COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Aug 5, 2026	**DATE SUBMITTED:**	Aug 10, 2026
NAME:	Bal, Kevin Ranz J.	**INSTRUCTOR:**	Engr Jimlord Quejado

Hands-on Activity 4.1: Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

PROCEDURE:

Procedure that performed:

ILO A

Task Title & Task Objective

Definition from the official CPP documentation for the stack STL.
Stacks are a type of container adaptor, specifically designed to operate in a LIFO context (last-in first-out), where elements are inserted and extracted only from one end of the container.
stacks are implemented as container adaptors, which are classes that use an encapsulated object of a specific container class as its underlying container, providing a specific set of member functions to access its elements. Elements are pushed/popped from the "back" of the specific container, which is known as the top of the stack. The underlying container may be any of the standard container class templates or some other specifically designed container class. The container shall support the following operations:

Screenshot of the code

stack_LL.h | stackArr.h | main.cpp

HoA 4.1 > ILO A > main.cpp > Inventory

```cpp
#include <iostream>
#include <stack>

// Emplace Struct Function
struct Inventory
{
    std::string attributes;
    std::string name;
    Inventory(std::string n, std::string a) : name(n), attributes(a) {} // constructor for the data inside the struct
};

int main()
{
    // push
    // pop
    // isEmpty
    // isFull
    // Top

    std::stack<char> stack1;

    std::cout << "is the stack empty? " << stack1.empty() << std::endl;
    stack1.push('j');
    std::cout << "current top of stack " << stack1.top() << std::endl;
    stack1.push('i');
    std::cout << "current top of stack " << stack1.top() << std::endl;
    stack1.push('m');
    std::cout << "current top of stack " << stack1.top() << std::endl;
    std::cout << "is the stack empty?" << stack1.empty() << std::endl;
    std::cout << "what is the size of the stack " << stack1.size() << std::endl;

    // declaration stack for two variable
    std::stack<int> xen1;
    std::stack<int> xen2;

    // initialization of what number value inside between stack 1 and 2
    xen1.push(90);
    xen2.push(60);
    // checking
    std::cout << "your current number stack on xen1 is " << xen1.top() << std::endl;
    std::cout << "your current number stack on xen2 is " << xen2.top() << std::endl;

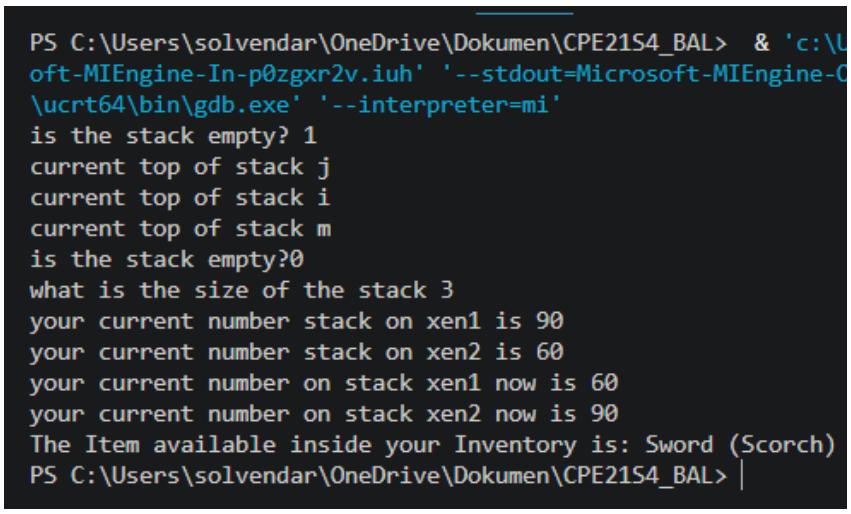
    // swap function
    xen1.swap(xen2);
    // check if the swap was complete
    std::cout << "your current number on stack xen1 now is " << xen1.top() << std::endl;
    std::cout << "your current number on stack xen2 now is " << xen2.top() << std::endl;
```

```cpp
    std::cout << "your current number on stack xen2 now is " << xen2.top() << std::endl;

    // emplace
    std::stack<Inventory> item;
    // DEFINING
    item.push(Inventory("Sword", "Scorch"));
    // CALLING THE EMPLACE
    item.emplace(Inventory("Sword", "Scorch"));

    std::cout << "The Item available inside your Inventory is: " << item.top().name << " (" << item.top().attributes << ")" << std::endl;

    return 0;
}
```

Output of the code	
Analysis	This output contains multiple operations using stack STL and the empty stack checks in 1 or 0, meaning if it's true or false. It shows the top of the stack was the "j i m" but in order, "m" will be the top and the "j" will be the last because of the LIFO principle. And also size function checking the size of the stack and it based on the push it does and also Pop function where when it is called the top will be removed and what below the top will be the current top. Swap function is swapping the value of the two variables by calling the first variable and will be called via .swap() and inside the parentheses will be the another variable that will be swapped to the value of the first variable.

PROCEDURE:

ILO B.1

Task Title & Task Objective
Stacks using Arrays

Screenshot of the code Static Array Implementation

HoA 4.1 > ILO B > main2.cpp > isFull()

```cpp
#include <iostream>
#include <stack>

// DEFINITION CHOICE
// #define maxSize 10 (method 1)
// const int maxSize = 10; (method 2)
// const maxSize_t = 10;

// global variable
#define maxSize 10
int stackArr[maxSize];
int top = -1, newData;

// function prototype
// Top, Push, isEmpty, isFull

// observer
bool isEmpty();
bool isFull();
void Top(); // T always capital
void DisplayAll();

// transformer operators
void push();
void pop();

int main()
{

    while (true)
    {

        int choice;

        std::cout << "============================================\n";
        std::cout << "               Stack Operations: " << std::endl;
        std::cout << "[1] push" << std::endl;
        std::cout << "[2] pop" << std::endl;
        std::cout << "[3] isEmpty" << std::endl;
        std::cout << "[4] isFull" << std::endl;
        std::cout << "[5] Top" << std::endl;
        std::cout << "[6] Display All" << std::endl;
        std::cout << std::endl;
        std::cout << "Input your choice ";
        std::cin >> choice;

        switch (choice)
        {
```

HoA 4.1 > ILO B > main2.cpp > isFull()

```
27  int main()
30      while (true)
45          std::cin >> choice;
46
47          switch (choice)
48          {
49
50          case 1:
51              push();
52              break;
53          case 2:
54              pop();
55              break;
56          case 3:
57              isEmpty();
58              break;
59          case 4:
60              isFull();
61              break;
62          case 5:
63              Top();
64              break;
65          case 6:
66              DisplayAll();
67              break;
68          default:
69              std::cout << "Invalid choice." << std::endl;
70          }
71      }
72      return 0;
73  }
74  // function definition
75  bool isEmpty()
76  {
77      return (top == -1);
78  }
79
80  bool isFull()
81  {
82      return (top == maxSize - 1);
83  }
84
85  void Top()
86  {
87
88      // prints the top element
89      std::cout << "The top element is: " << stackArr[top] << std::endl;
90  }
91
```

```
void push()
{

    // error catching
    if (isFull())
        std::cout << "Stack Overflow!" << std::endl;
    return;

    // ask the user what they want to insert:
    std::cout << "what do you want to push? ";
    std::cin >> newData;

    // storing to the stack
    stackArr[++top] = newData;
}
void pop()
{

    // error catching
    if (isEmpty())
        std::cout << "Stack Underflow!" << std::endl;
    return;

    // check if empty
    std::cout << "Popping " << stackArr[top] << std::endl;

    // to remove the element from the stack
    top--;
}

void DisplayAll()
{

    // error checking
    if (isEmpty())
        std::cout << "The Stack is empty" << std::endl;
    return;

    // displaying the stack and the LIFO principle
    std::cout << "The Elements of Stack from top to bottom " << std::endl;
    for (int i = top; i >= 0; i--)
    {
        std::cout << stackArr[i] << std::endl;
    }
}
```

Output of the code

```
PS C:\Users\solvendar\OneDrive\Dokumen\CPE21S4_BAL>  & 'c:\
oft-MIEngine-In-cqj5wtkg.foe' '--stdout=Microsoft-MIEngine-
\ucrt64\bin\gdb.exe' '--interpreter=mi'
================================================
                Stack Operations:
[1] push
[2] pop
[3] isEmpty
[4] isFull
[5] Top
[6] Display All

Input your choice 1
================================================
                Stack Operations:
[1] push
[2] pop
[3] isEmpty
[4] isFull
[5] Top
[6] Display All

Input your choice 2
Stack Underflow!
================================================
                Stack Operations:
[1] push
[2] pop
[3] isEmpty
[4] isFull
[5] Top
[6] Display All

Input your choice 3
================================================
                Stack Operations:
[1] push
[2] pop
[3] isEmpty
[4] isFull
[5] Top
[6] Display All

Input your choice 4
================================================
                Stack Operations:
[1] push
[2] pop
[3] isEmpty
[4] isFull
[5] Top
[6] Display All

Input your choice 5
The top element is: 0
```

```
==================================================
                Stack Operations:
[1] push
[2] pop
[3] isEmpty
[4] isFull
[5] Top
[6] Display All

Input your choice 6
The Stack is empty
==================================================
                Stack Operations:
[1] push
[2] pop
[3] isEmpty
[4] isFull
[5] Top
[6] Display All

Input your choice 7
Invalid choice.
```

Analysis	This output contains or belong to the array implementation of the stack that has a five options to pick which is push, pop, isEmpty, isFull, and Display All that used a switch case based on the user's choice of which function will be executed

ILO B.2

Task Title & Task Objective

Stacks using Linked Lists

Screenshot of the code Linked List Implementation (Header)	

HoA 4.1 > ILO B > stacklist.h > pop<T>()

```
1   #ifndef STACK_LIST_H
2   #define STACK_LIST_H
3
4   // class node that contains data and the next
5   template <typename T>
6   class Node
7   {
8   public:
9       T data;     // VALUE THAT WE NEED STORE
10      Node *next; // POINTER TO THE NEXT NODE
11  };
12
13  // global pointer for the top of the stack
14  template <typename T>
15  Node<T> *head = nullptr;
16
17  // operations for the stack
18  //  push
19  template <typename T>
20  void push(T newData)
21  {
22      // create a new node and assign the newData
23      Node<T> *newNode = new Node<T>;
24      newNode->data = newData;
25
26      // new node points to current top, then becomes the new top
27      newNode->next = head<T>;
28      head<T> = newNode;
29  }
30
31  // pop
32  template <typename T>
33  T pop()
34  {
35      // check if the stack is empty:
36      if (head<T> == nullptr)
37      {
38          std::cout << "stack overflow. " << std::endl;
39      }
40
41      // save the current top node
42      Node<T> *temp = head<T>;
43      // store its data
44      T tempVal = temp->data;
45      // move the head to the next node
```

```
T pop()
    // move the head to the next node
    head<T> = head<T>->next;
    // free the removed node
    delete temp;
    // return the removed value
    return tempVal;
}

// isEmpty
template <typename T>
bool isEmpty()
{
    return head<T> == nullptr;
}

// top
template <typename T>
void Top()
{
    // error catching:
    if (isEmpty<T>())
    {
        std::cout << "Stack underflow " << std::endl;
        return;
    }
    // display the Top node
    std::cout << "Top of Stack: " << head<T>->data << std::endl;
}

// display All
template <typename T>
void displayALL()
{
    if (head<T> == nullptr)
    {
        std::cout << "stack undeflow\n";
        return;
    }
    std::cout << "Displaying the stack: \n";
    Node<T> *temp = head<T>;
    while (temp != nullptr)
    {
        std::cout << temp->data << "\n";
        temp = temp->next;
    }
    std::cout << "\n";
}

#endif
```

Main Driver Code

HoA 4.1 > ILO B > main2.cpp > isFull()

```cpp
#include <iostream>
#include <stack>

// DEFINITION CHOICE
// #define maxSize 10 (method 1)
// const int maxSize = 10; (method 2)
// const maxSize_t = 10;

// global variable
#define maxSize 10
int stackArr[maxSize];
int top = -1, newData;

// function prototype
// Top, Push, isEmpty, isFull

// observer
bool isEmpty();
bool isFull();
void Top(); // T always capital
void DisplayAll();

// transformer operators
void push();
void pop();

int main()
{

    while (true)
    {

        int choice;

        std::cout << "================================================\n";
        std::cout << "                Stack Operations: " << std::endl;
        std::cout << "[1] push" << std::endl;
        std::cout << "[2] pop" << std::endl;
        std::cout << "[3] isEmpty" << std::endl;
        std::cout << "[4] isFull" << std::endl;
        std::cout << "[5] Top" << std::endl;
        std::cout << "[6] Display All" << std::endl;
        std::cout << std::endl;
        std::cout << "Input your choice ";
        std::cin >> choice;
```

```
30      while (true)
47          switch (choice)
49
50          case 1:
51              push();
52              break;
53          case 2:
54              pop();
55              break;
56          case 3:
57              isEmpty();
58              break;
59          case 4:
60              isFull();
61              break;
62          case 5:
63              Top();
64              break;
65          case 6:
66              DisplayAll();
67              break;
68          default:
69              std::cout << "Invalid choice." << std::endl;
70          }
71      }
72      return 0;
73  }
74  // function definition
75  bool isEmpty()
76  {
77      return (top == -1);
78  }
79
80  bool isFull()
81  {
82      return (top == maxSize - 1);
83  }
84
85  void Top()
86  {
87
88      // prints the top element
89      std::cout << "The top element is: " << stackArr[top] << std::endl;
90  }
```

```
void push()
{

    // error catching
    if (isFull())
        std::cout << "Stack Overflow!" << std::endl;
    return;

    // ask the user what they want to insert:
    std::cout << "what do you want to push? ";
    std::cin >> newData;

    // storing to the stack
    stackArr[++top] = newData;
}
void pop()
{

    // error catching
    if (isEmpty())
        std::cout << "Stack Underflow!" << std::endl;
    return;

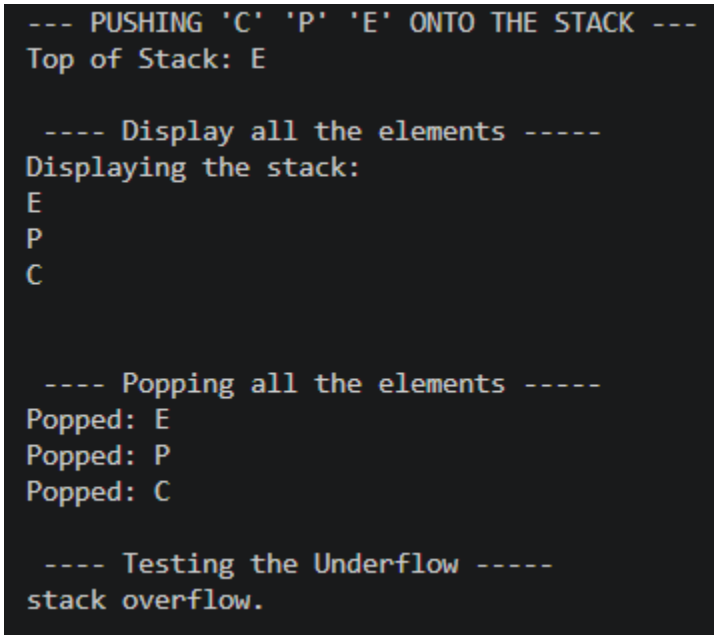
    // check if empty
    std::cout << "Popping " << stackArr[top] << std::endl;

    // to remove the element from the stack
    top--;
}

void DisplayAll()
{

    // error checking
    if (isEmpty())
        std::cout << "The Stack is empty" << std::endl;
    return;

    // displaying the stack and the LIFO principle
    std::cout << "The Elements of Stack from top to bottom " << std::endl;
    for (int i = top; i >= 0; i--)
    {
        std::cout << stackArr[i] << std::endl;
    }
}
```

Output of the code	
Analysis	This header contains template functions that display the stacks using linked list implementation and these functions are called in the main drive and called the header inside of the driver so the functions that are created inside the header will be called and used inside. The push function was the one creating new nodes and assigning data which the new node will become the top node. While the pop function deletes a specific node from the stack and saves the current top stack and stores the data of it and if there are no nodes or the stack is empty, it will return and say the stack is underflow. isEmpty function was a boolean where it checks whether the head of the node is empty or not. It returns a true value and else is false. The top function also uses boolean just like isEmpty to check if the stack is empty, while this one uses the current head node that data is on top of the stack and displays it. displayAll, will check the contents that similar to pop if statement and makes a temporary variable that store the data that will be the output and uses a next pointer to traverse it with the nodes

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

SUPPLEMENTARY:

ILO C

Task Title & Objective:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:
a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL

Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {,
or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:
1. Create a Stack.
2. While(end of input is not reached) {
a) If the character read is not a symbol to be balanced, ignore it.
b) If the character is an opening symbol, push it onto the stack.
c) If it is a closing symbol:
i) Report an error if the stack is empty.
ii) Otherwise, pop the stack.
d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

Screenshot of the code
Stack Array Header

HoA 4.1 > ILO C > C stackArr.h > ...

```
1   #ifndef STACKARR_H
2   #define STACKARR_H
3   #include <iostream>
4
5   const int maxSize = 10;
6   char stackArr[maxSize];
7   int top = -1;
8
9   // obsevers
10  bool isFull();
11  bool isEmpty();
12  void Top();
13
14  // transformer
15  void pop();
16  void push(char newData); // to accept character parameter
17
18  bool isEmpty()
19  {
20      return (top == -1);
21  }
22
23  bool isFull()
24  {
25      return (top == maxSize - 1);
26  }
27
28  void Top()
29  {
30      if (isEmpty())
31      {
32          std::cout << "Stack is Empty" << std::endl;
33          return;
34      }
35      // else
36      std::cout << "The top element is: " << stackArr[top] << std::endl;
37  }
38
39  void push(char newData)
40  {
41
42      // error catching
43      if (isFull())
44      {
45          std::cout << "Stack Overflow!" << std::endl;
46          return;
47      }
48      // storing to the stack
49      stackArr[++top] = newData;
50  }
51
```

```
void pop()
{
    // error catching
    if (isEmpty())
    {
        std::cout << "Stack Underflow!" << std::endl;
        return;
    }
    // else
    // to remove the element from the stack
    top--;
}
#endif
```

Screenshot of the code Linked List Header

HoA 4.1 > ILO C > C stack_LL.h > ...

```
#ifndef STACK_LL_H
#define STACK_LL_H
#include <iostream>

template <typename T>
class sNode
{

public:
    T data;
    sNode *next;
};

// global top/head pointer for tracking the stack
sNode<char> *head = nullptr;

// Observers
bool list_Empty();
void list_Top();

// Transformers
void list_push(char newData);
void list_pop();

// allocate a new node
template <typename T>
sNode<T> *new_node(T newData)
{

    sNode<T> *newNode = new sNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

// function implementation
bool list_Empty()
{
    return (head == nullptr);
}

void list_Top()
{
    if (list_Empty())
    {
        std::cout << "Stack is Empty" << std::endl;
        return;
    }
    std::cout << "The top element is: " << head->data << std::endl;
}

```

```
void list_push(char newData)
{
    sNode<char> *newNode = new_node(newData);

    // set the new node next to point to the current head
    newNode->next = head;

    // update the head pointer to become this new node
    head = newNode;
}

void list_pop()
{
    if (list_Empty())
    {
        std::cout << "Stack Underflow!" << std::endl;
        return;
    }

    // keep track of the node that want to delete
    sNode<char> *temp = head;

    // advance the head pointer to the next lower node
    head = head->next;

    // delete the memory of the popped node
    delete temp;
}

#endif
```

Main Driver Code

stack_LL.h | stackArr.h | suppstack.cpp

HoA 4.1 > ILO C > suppstack.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "stackArr.h"
#include "stack_ll.h"

// checker for array implementation
bool checkArr(std::string expr)
{
    top = -1; // reset global array tracker for every new test string

    for (char ch : expr)
    {
        if (ch == '(' || ch == '{' || ch == '[')
        {
            push(ch); // uses array header push
        }
        else if (ch == ')' || ch == '}' || ch == ']')
        {
            if (isEmpty())
                return false;

            char open = stackArr[top]; // read from the global array header
            pop();                     // uses array header pop

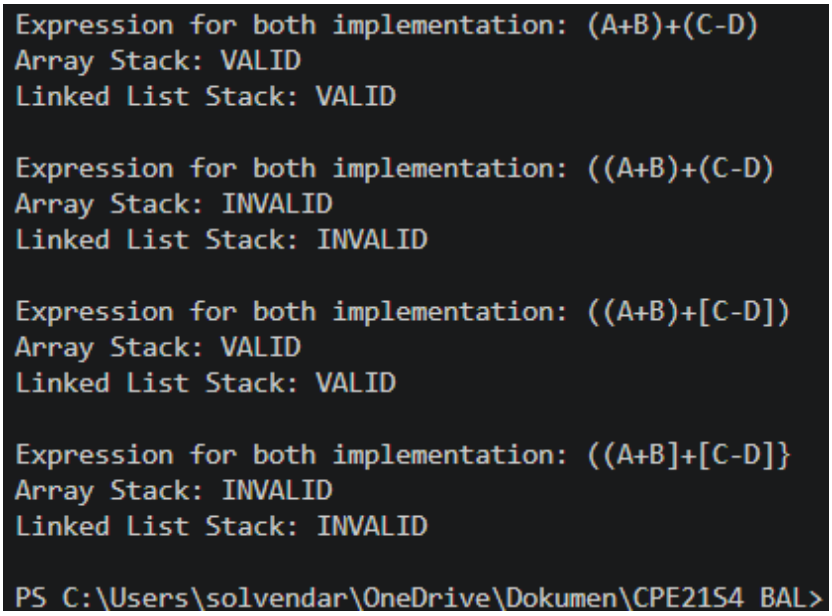
            if (ch == ')' && open != '(')
                return false;
            if (ch == '}' && open != '{')
                return false;
            if (ch == ']' && open != '[')
                return false;
        }
    }
    return isEmpty();
}

// checker for linked list implementation
bool checkList(std::string expr)
{
    // clear the remaining nodes in the global head pointer
    while (!list_Empty())
    {
        list_pop();
    }
    head = nullptr;

    for (char ch : expr)
    {

        if (ch == '(' || ch == '{' || ch == '[')
        {
            list_push(ch); // uses linked list header push
```

```
52          }
53          else if (ch == ')' || ch == '}' || ch == ']')
54          {
55              if (list_Empty())
56                  return false;
57
58              char open = head->data; // read from the global head node in the header
59              list_pop();             // used the linked list header pop
60
61              if (ch == ')' && open != '(')
62                  return false;
63              if (ch == '}' && open != '{')
64                  return false;
65              if (ch == ']' && open != '[')
66                  return false;
67          }
68      }
69      return list_Empty();
70  }
71
72  int main()
73  {
74
75      std::string expressions[] = {
76          "(A+B)+(C-D)",
77          "((A+B)+(C-D)",
78          "((A+B)+[C-D])",
79          "((A+B]+[C-D]}"};
80
81      for (std::string expr : expressions)
82      {
83          std::cout << "Expression for both implementation: " << expr << "\n";
84
85          // array stack implementation
86          if (checkArr(expr))
87          {
88              std::cout << "Array Stack: VALID\n";
89          }
90          else
91          {
92              std::cout << "Array Stack: INVALID\n";
93          }
94          // linked list stack implementation
95          if (checkList(expr))
96          {
97              std::cout << "Linked List Stack: VALID\n";
98          }
99          else
100         {
101             std::cout << "Linked List Stack: INVALID\n";
102         }
103         std::cout << "\n";
104     }
105
106     return 0;
107 }
```

Output of the code	
Analysis	In my array implementation, the checker function loops through the characters of the inputted std::string expr string variable. Before reading the string, I make sure to reset the global integer tracker top to -1 so each new expression starts fresh and empty. When my loop finds an opening bracket, it calls the push function from my header file which adds 1 to top and stores the character into my stackArr array. When a closing bracket arrives, my code reads the top element directly from the global array index using stackArr[top] and calls pop to subtract 1 from the counter. If a closing bracket appears while isEmpty() is true or if the closing bracket does not match the saved character, the function stops right away and returns false.Lastly, if the loop finishes and isEmpty() confirms that top equals -1, it proves that all symbols matched perfectly and the expression prints out as valid. For my linked list implementation, I use the exact same loop and check structure as the array but manage memory directly through pointers. I just reuse a previous code and modified it named sNode which allows me to create new nodes specifically for characters by using my new_node function. Before reading an expression, I run a loop that calls list_pop() until list_Empty() is true to reset my global head pointer back to null. When an opening bracket is hit, my code creates a new node, points its next pointer directly to the current head, and updates the head pointer to be this new node. When a closing bracket comes up, my program checks if it is empty using list_Empty(), reads the character using head->data, and moves the head pointer to the next node using head->next. The old node is then deleted to clear the memory, and a successful final check showing that head returned back to nullptr means the brackets are completely valid.

How do the different internal representations affect the implementation and usage of the stack?	In my experience with this activity, I noticed that the way the stack is set up on the inside completely changes how it uses memory and handles size limits. For my array stack, the code was simple to write because it just uses a fixed box of memory with a set limit. It saves memory because I do not need to use pointers, but I am stuck with a maximum size and will get a stack overflow error. On the other hand, my linked list stack uses memory addresses and pointers to connect data blocks one by one. Writing this code as my header took me a while because I had to create and delete memory blocks carefully, but using it is much better because it can grow as much as I need and never runs out of space
Summary of lessons learned	I learned how to make and use a stack in two totally different ways. I now understand that an array uses a simple number tracker to know where the top is, while a linked list uses connected blocks and pointers to hold the data.
Analysis of the procedure	It depends on the person what he/she used when it comes to stack if it's linked list or array implementation because both of them can be used and also at the same time but their implementations are different and also handle their data. And for me, it's hard for me till now to handle linked lists well and need to look at the previous procedure codes that are given just to remember and use it as my reference when doing this. Array Implementation requires access to the elements of the same data type, while Linked List Implementation uses a node to store elements in different memory locations and need to find a way to traverse the nodes.
Analysis of the supplementary activity	The bracket checking task showed me that both of my methods give the exact same correct results even though they work differently behind it. My array stack worked fine with its fixed layout, and my template linked list well made and deleted nodes as the code ran. Testing the four specific expressions from my supplementary activity showed that correct brackets pass easily, while wrong or extra brackets can see right away because my stack fails to end up completely empty.
Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?	For me, it's not bad to reuse the previous Hands-on if it can visualize it when it comes to supplementary and see a given code from the procedure that is similar to it and all I do was to re-modify it and also search about implementation when it comes to both headers. All I needed was to memorize every implementation of header and linked list when it comes to stack and queue and how to handle their data well mostly in linked list header.

Self-Checking: For the following cases, complete the table using the code you created.	Valid? (Y/N)	Output of the console each case	Analysis
(A+B)+(C-D)	Y	\ucrt64\bin\gdb.exe --interpreter=mi Expression for both implementation: (A+B)+(C-D) Array Stack: VALID Linked List Stack: VALID	There is a proper opening and closing parentheses that giving a reason to be considered as valid when it checked
((A+B)+(C-D)	N	Expression for both implementation: ((A+B)+(C-D) Array Stack: INVALID Linked List Stack: INVALID	There's an extra opening on the right side of the parenthesis and the left one was left alone and will be considered as invalid
((A+B)+[C-D])	Y	Expression for both implementation: ((A+B)+[C-D]) Array Stack: VALID Linked List Stack: VALID	It has a proper opening and closing bracket inside and also the outside was also closed, so it will be considered as valid
((A+B]+[C-D]}	N	Expression for both implementation: ((A+B]+[C-D]} Array Stack: INVALID Linked List Stack: INVALID	This expression is invalid because on the left side was an opening parenthesis, but on the right side it was a curly bracket instead of closing parenthesis.

Conclusion

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chrome Search (Gemini)**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, Idea Generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explanation of deep concepts about linked list implementation when it comes to stack without STL, guidance and generating clearer example concepts to visualize implementation of the linked list header part of this Hands-on Activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.